

Databricks Platform Security Docs

Databricks AI Features Security and Governance



Table of contents

Overview..... 3

Shared Design Principles..... 3

 All data is encrypted..... 3

 Objects have strong governance..... 3

 Models themselves are stateless..... 3

 We do not train GenAI models on your data..... 4

 Usage data is protected..... 4

 Databricks staff access is controlled..... 4

 Data residency controls are in place..... 4

Model Selection..... 5

Data Used in AI Features..... 6

 Example Scenario..... 6

 User Inputs..... 6

 Metadata..... 7

 Relevant Values..... 7

Architecture..... 7

 Model-Specific Architecture..... 8

Shared Responsibility for Managed AI Features..... 9

Feature-Specific Details..... 10

 Genie Agents..... 10

 Adding Table to Genie Agent..... 11

 Genie Agents Query..... 12

 Genie Code..... 13

 Genie Code Question..... 14

 AI/BI Dashboards..... 14

 Asking AI/BI Dashboard to generate a chart..... 15

Revision history..... 15

Overview

The goal of this document is to provide a high-level overview of the security and governance of the AI features in Databricks. The overall design and principles for these systems are very similar, and will be covered in the initial section. We will then break out service-specific details in later sections.

Note that many details in these later sections are implementation details and are subject to change, but we will strive to reflect the underlying principles that we adhere to and to provide the most reliable information available.

This document is a deep dive into the data flow and security capabilities, and should be used as an appendix to the Databricks AI Trust and Safety documentation ([AWS](#), [Azure](#), [GCP](#)). That documentation page includes details such as:

- Zero Data Retention configuration for our model partner (such as Azure OpenAI)
- Guarantees that Databricks will not train generative foundation models on your data
- Content filtering and testing strategies for mitigating harmful or insecure LLM responses (filtering levels will vary)
- Use of Databricks-hosted and third-party models, with control over model choice and metadata handling
- A detailed table mapping AI features to trust behavior, model host, and data-sharing settings

Scope: This document refers to the managed AI features built into the Databricks platform, including [Genie Agents](#), [Genie Code](#), and [AI/BI Dashboards](#). It does **not** apply to the capabilities used to build your own AI applications on top of the platform, such as Model Serving or Agent Bricks. As those features are very different, the security design for them is also very different. Please see the data protection section of our Model Serving documentation for more detail ([AWS](#), [Azure](#), [GCP](#)).

Shared Design Principles

Databricks reuses existing infrastructure components where possible to reduce complexity and maintain a uniform security model. As a result, many core security controls are shared across all of these features. Those include:

All data is encrypted

Data is stored encrypted at rest (AES-256 or greater) and in transit (TLS 1.2 or greater). Long term data storage (such as the Genie Agents database, AI/BI Dashboards, or Genie Code history) is included in our [Customer-managed keys: managed services feature](#).

Objects have strong governance

All objects have ACLs, or are private to the specific users/groups shared to. Your datasets are governed by your Unity Catalog policies. We do not send information to the models the current user does not have access to.

Models themselves are stateless

An LLM model itself ([we use](#) Azure OpenAI, Anthropic, and/or Databricks internally-hosted models) is stateless, and does not store any data. Even for very long conversations, each request to the LLM will include the entire conversation until that point. (This is not specific to Databricks, Azure OpenAI or any other provider – this is a trait of LLM models.)

We do not train GenAI models on your data

In addition to models being [stateless](#), our model partners do not retain data you submit through these features, even for abuse monitoring. Our partner-powered AI assistive features use zero data retention endpoints from our model partners. Databricks does not train generative foundation models with data you submit to these features. Please refer to our [AI Trust and Safety documentation](#) for more details.

Usage data is protected

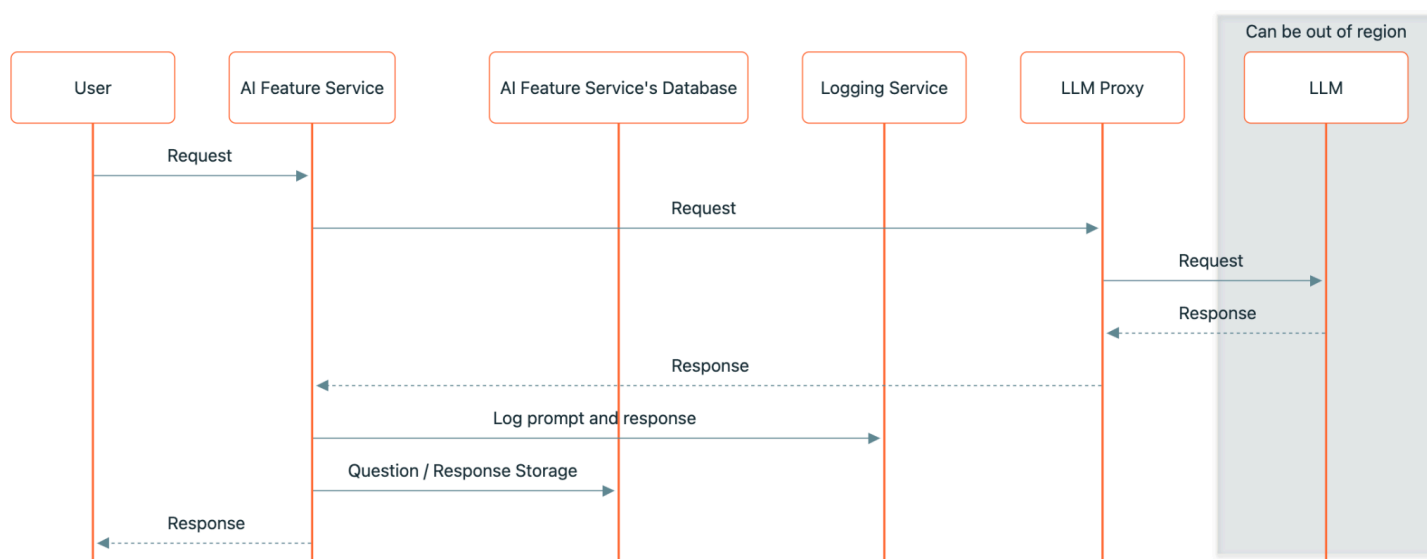
For these features, user and system inputs (such as metadata and row-level data specific to the question at hand) and outputs (such as the generated prompt text) are logged and retained for 30 days. Only Databricks staff whose job requires access will have access (such as support, engineers, or security). The logs are only available via an internal Databricks workspace requiring SSO+MFA, VPN, and must obtain internal approval to be able to access this data for debugging purposes. Audit logs for those workspaces are centralized for the Databricks SIEM.

Databricks staff access is controlled

Production systems supporting these features utilize the same just-in-time authentication and authorization as all Databricks services.

Data residency controls are in place

In certain cases that we outline in our documentation ([AWS](#), [Azure](#), [GCP](#)), AI Features may call beyond the local cloud or geo if cross-geo processing is enabled ([AWS](#), [Azure](#), [GCP](#)). For example, many AI Features use Azure OpenAI, which is available in multiple regions in the Azure cloud but not in other clouds. When this occurs, the request is sent to a Databricks-owned LLM proxy in the source control plane, which can send directly to the model in the destination region. The destination region does not perform any logging, [maintain any state](#), or create any record of the request. Logging and usage data is handled just as with other Databricks services.



Model Selection

Many customers want to understand which hosting provider each feature uses. The authoritative copy of this information comes from our Databricks Partner-powered AI features documentation ([AWS](#), [Azure](#), [GCP](#)). For convenience, we have included an extract below. Where the providers differ by cloud, the table calls out AWS/GCP and Azure separately. Databricks regularly tests alternative models, including models from other frontier providers, and may shift features to better-performing models.

Feature	Hosting provider when partner-powered AI features is enabled	Hosting provider when partner-powered AI features is disabled
Genie Code Autocomplete	Databricks	Databricks
Genie Code Agent mode	AWS/GCP: Azure OpenAI service, OpenAI on Databricks, and Anthropic on Databricks† Azure: Azure OpenAI service and Anthropic on Databricks†	Feature is not available
Genie Code (and all other Assistant subfeatures)	AWS/GCP: Azure OpenAI service and OpenAI on Databricks Azure: Azure OpenAI service	Databricks for non-CSP workspaces. For CSP workspaces, availability depends on the compliance standard.‡
AI/BI dashboard AI-assisted visualizations and companion Genie Agents (Public Preview)	Azure OpenAI service	Feature is not available
Genie Agents Chat mode	AWS/GCP: Azure OpenAI service and OpenAI on Databricks Azure: Azure OpenAI service	Feature is not available
Genie Agents Agent mode	AWS/GCP: OpenAI on Databricks and Anthropic on Databricks† Azure: Anthropic on Databricks†	Feature is not available
Genie One	AWS/GCP: OpenAI on Databricks and Anthropic on Databricks† Azure: Anthropic on Databricks†	Chat is not available. Users can still access Genie One and interact with data assets.
AI-generated comments	Azure OpenAI service	Databricks
LLMs as judges *	Azure OpenAI service	Users must specify the model to use
Synthetic data generation *	Azure OpenAI service	Feature is not available

* These features are not [AI assistive features](#), but require the partner-powered AI features setting.

† Anthropic on Databricks uses endpoints hosted by Databricks within the Databricks security perimeter.

‡ When partner-powered AI features are disabled, Genie Code availability in CSP workspaces depends on the cloud, region, and compliance standard; see the [linked live documentation](#).

From the [Databricks Partner-powered AI features documentation](#); checked September 10, 2026.

Data Used in AI Features

Data from your workspace will be used when interacting with our AI Features (see our documentation: [AWS](#), [Azure](#), [GCP](#)). This falls into three categories: User Inputs (such as free-text questions), Metadata (such as table schemas), Relevant Values (such as sampled values or prior results).

The [shared design principles](#) described earlier apply to all of these categories in the same way – Databricks does not try to differentiate security controls based on the category (or even sources of data within a category). Values from your data may show up in sampled values, prior query results, and users may paste them in free-text queries – we provide the same level of protection across all of those scenarios.

To help illustrate these categories, please see the example scenario below for Genie Agents. An example based on Genie Code would also be functionally similar.

Example Scenario

For our example, we will use a Genie Agents at an auto dealership chain. The space is set up with:

1. A table of regional car sales (month, dealer_name, make, model, num_sold, dealer_revenue, list_price)
2. Example queries that users might run against the space.
3. Guidance to provide useful queries (like “Users may ask about quarters. Q1 starts Feb 01, Q2 Apr 01, etc.”)

A business analyst has the following conversation with a Genie Agent:

- Analyst: What dealerships are selling the most Ford F150s this quarter, and at what discount rate?
- Genie Agent: <Generates and runs a SQL query like the following, and provides a table of values>

```
select
    dealer_name,
    sum(dealer_revenue) total_dealer_revenue,
    sum(dealer_revenue) / sum(list_price) discount_percent
from sales
where
    make = 'Ford' and model='F-150' and month >= '2025-04-01'
```
- Analyst: Has that #1 seller always discounted more than the others?
- Genie Agent: <Analyzes the previous table of values to determine who the #1 seller is, builds a new query to provide the answer>

User Inputs

All of our AI features allow users to provide free-text input (like questions or prompts). In our example above, the user inputs would be the questions the user provided (“What dealerships are selling the most Ford F150s this quarter?”) along with the configuration the space owner provided (guidance like “Users may ask about quarters. Q1 starts Feb 01, Q2 Apr 01, etc.”)

Metadata

When data is added to a Genie Agent, metadata is automatically pulled to help the model generate useful queries. In our example, this would include the sales table details (such as column names, descriptions, primary key / foreign key relationships). The queries added to the space may also be included (just as favorite queries may be included when using Genie Code).

Relevant Values

There are a variety of scenarios where relevant data is included in order to allow the Genie Agent to function properly. Two good examples are prompt matching, and referencing prior results.

[Prompt matching](#) (formerly known as Data Sampling or Value Sampling): The space owner can enable prompt matching to allow the model to build accurate queries. Without prompt matching, the model might not correctly split make and model when the user asks for “Ford F150” and wouldn’t provide an accurate result when the database value is “F-150” but the user asks for “F150.” These values are included in a control plane database and/or [workspace system data](#), and subject to CMK if enabled ([AWS](#), [Azure](#), [GCP](#)).

Referencing prior results: there are many instances where the user will want to ask follow-ups or clarifications on the prior response. In our example, this would include the user asking about the #1 seller (requiring the model to know who the seller is in that table). This context is also essential when the user asks the Genie Agent to fix an issue in the results, or if the user asks the Genie Agent to further analyze the result (such as determine anomalies).

Databricks continues to evolve how we use relevant values, keeping in line with comparable tools in the marketplace, but we will maintain the same overall security model and categories of data.

Architecture

AI/BI Genie separates user interaction, model inference, and SQL execution across distinct service boundaries. The diagram below illustrates the components involved, which fall into three broad steps. Please note that we have [state diagrams below](#), which can also help explain the step-by-step process.

User Request

A request enters through the Databricks web application or REST API and is handled by the AI/BI Genie service in the control plane. The service stores the application state required to operate Genie and sends model requests through a Databricks-managed LLM proxy.

LLM Inference

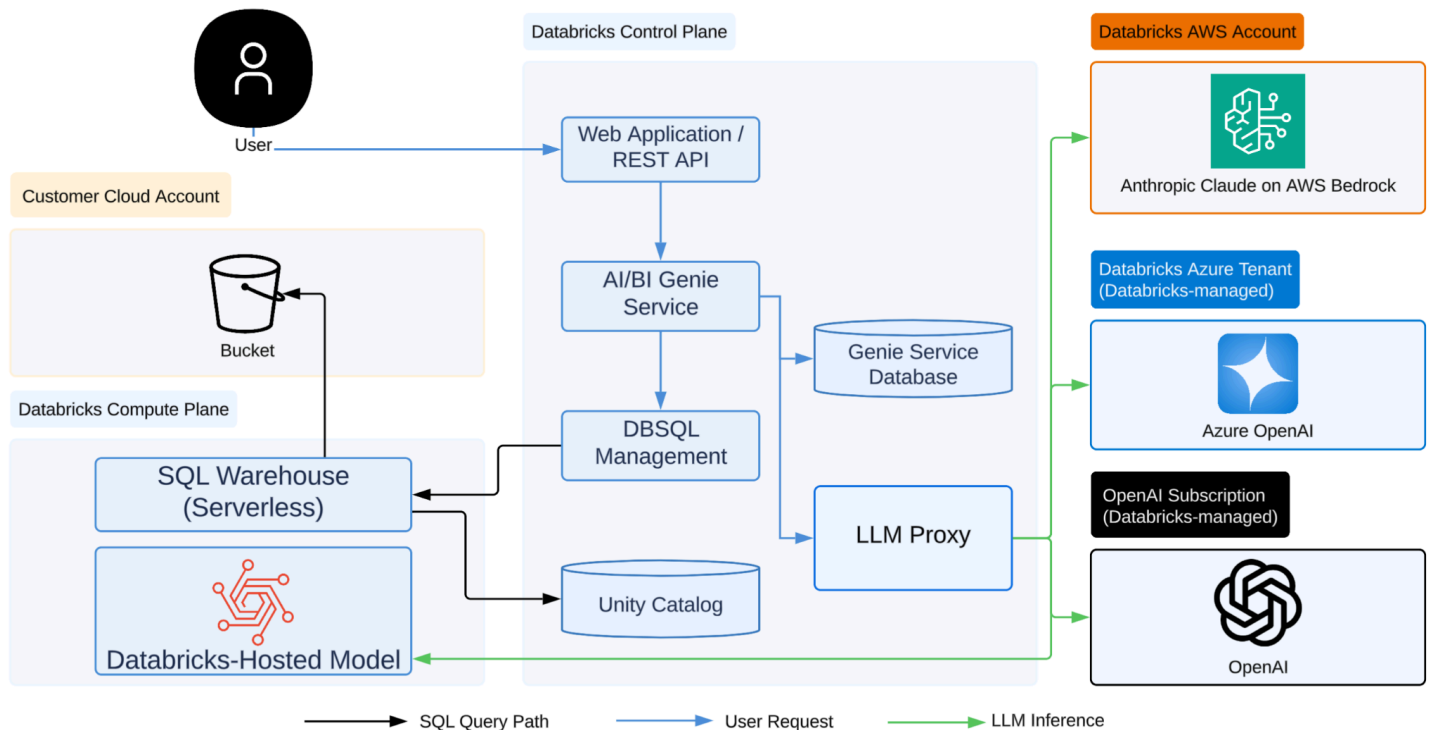
The proxy routes each inference request to a supported Databricks-hosted or partner-powered model. The route depends on the feature, cloud, geography, and customer configuration. The diagram illustrates possible routes rather than guaranteeing that every provider is available for every configuration.

Model inference and SQL execution follow separate paths. The model can receive the user's input, authorized metadata, and relevant values needed to generate a response or candidate query. It does not connect directly to customer storage or execute SQL.

SQL Query Execution

Genie submits queries through Databricks SQL, and the serverless SQL warehouse uses Unity Catalog to enforce the requesting user's permissions before accessing customer data. While most elements of our AI Features rely on multi-tenant services, the actual query execution uses the existing Databricks SQL structure where warehouses are dedicated to just one customer.

This separation preserves the platform's existing authorization boundaries. Customers control identities, Unity Catalog permissions, the data made available to Genie, and applicable Partner-Powered AI settings. Databricks secures the managed services, model integrations, and query execution path. Model providers operate their endpoints under their agreements with Databricks. Because model outputs can be incomplete or incorrect, customers should review generated queries and responses before relying on them.



High-level architecture for the Genie service

Model-Specific Architecture

As shown above, Databricks AI features send model requests from the Databricks control plane through a Databricks-managed routing layer. The selected endpoint depends on the feature, workspace cloud and geography, account settings, and service availability. Because some providers operate in a particular cloud, model processing can occur in a different cloud from the workspace. Databricks Designated Services and cross-geography processing settings govern eligible processing locations. Built-in features use Databricks-managed provider credentials.

Azure OpenAI Service

Azure Databricks workspaces use Azure OpenAI endpoints hosted within Azure Databricks-managed subscriptions. Azure OpenAI endpoints can also serve Databricks workspaces hosted on AWS or Google Cloud. For these workspaces, requests are routed to Azure OpenAI endpoints hosted within Databricks-owned Azure subscriptions.

Anthropic on Databricks

Requests are routed to Anthropic models hosted by using AWS Bedrock within our Databricks AWS account, within the Databricks security perimeter. This architecture can serve workspaces on AWS, Azure, or GCP without requiring customers to provide an Anthropic or AWS account.

OpenAI on Databricks

Available only to workspaces on AWS and GCP, requests are routed to endpoints hosted in a Databricks tenant on OpenAI, managed and secured as part of the Databricks platform. Databricks manages the serving endpoints, routing, and provider credentials, so customers do not need to establish or manage their own OpenAI account. We may also consume OpenAI from Bedrock.

Databricks-hosted models

Databricks-hosted models use the same custom model-serving infrastructure that is available to customers, but in workspaces specifically designated for this purpose.

Shared Responsibility for Managed AI Features

Managed AI features use the platform controls described in the Databricks shared responsibility model. AI adds specific responsibilities for feature governance, data sent to a model, provider operations, and the use of generated output. For Partner-Powered AI features, Databricks selects and manages the model-provider integration as part of the Databricks service. Customers do not configure a separate provider account for those requests.

Model outputs are probabilistic and can be incomplete or incorrect. Databricks and its model providers operate the managed service and the model path. Customers remain responsible for deciding where AI is appropriate and for reviewing outputs before relying on them or using them in downstream actions.

Area	Customer responsibilities	Databricks responsibilities	Model-provider responsibilities
Feature governance	Approve the use cases, users, and data. Configure Partner-Powered AI and other available account or workspace controls based on organizational policy.	Provide and enforce documented controls. Respect the current user's permissions. Document current model hosting and data-sharing behavior.	Honor the configuration and terms established with Databricks. The provider does not decide which customer users, data, or use cases are authorized.

Area	Customer responsibilities	Databricks responsibilities	Model-provider responsibilities
Data handling	Classify and govern source data. Set Unity Catalog permissions and object ACLs. Control prompts, space instructions, and other submitted content.	Use data that the current user can access. Protect and isolate requests. Apply documented retention and training commitments across supported model routes.	Process requests to produce responses and meet the security, retention, and training restrictions agreed with Databricks.
Service operation	Secure customer-managed identities, data, and configurations. Enable available audit and monitoring controls, and investigate systems under customer control.	Secure the feature, LLM proxy, integrations, logs, and staff access. Select and oversee providers. Monitor and respond to incidents under Databricks control.	Secure and operate the model service and infrastructure under the provider's control. Notify Databricks of relevant incidents or material service changes as required by the applicable terms.
Outputs and actions	Review generated text, queries, code, and recommendations. Apply human oversight based on the use case. Remain responsible for downstream actions and compliance.	Design and test the feature. Apply safeguards where applicable. Maintain deterministic authorization and execution controls, and document material limitations.	Produce the model response and operate included model-level safety mechanisms. The provider does not guarantee correctness or decide how the customer uses the output.

When Databricks hosts the model, Databricks also carries the model-hosting responsibilities described in the model-provider column. Customer-built AI applications, customer-selected provider endpoints, Model Serving, and Agent Bricks are outside the scope of this document and follow their own product documentation and provider arrangements.

Feature-Specific Details

Below, see a collection of details and diagrams specific to individual features.

Genie Agents

There are two key elements to Genie: SQL generation and SQL execution. An LLM is used to translate user questions into SQL statements and textual output. Databricks sends your prompt, relevant table metadata and values, errors, as well as input code or queries as a part of that reasoning process. SQL Execution occurs on Databricks SQL warehouses, just like SQL executed via the Query Editor.

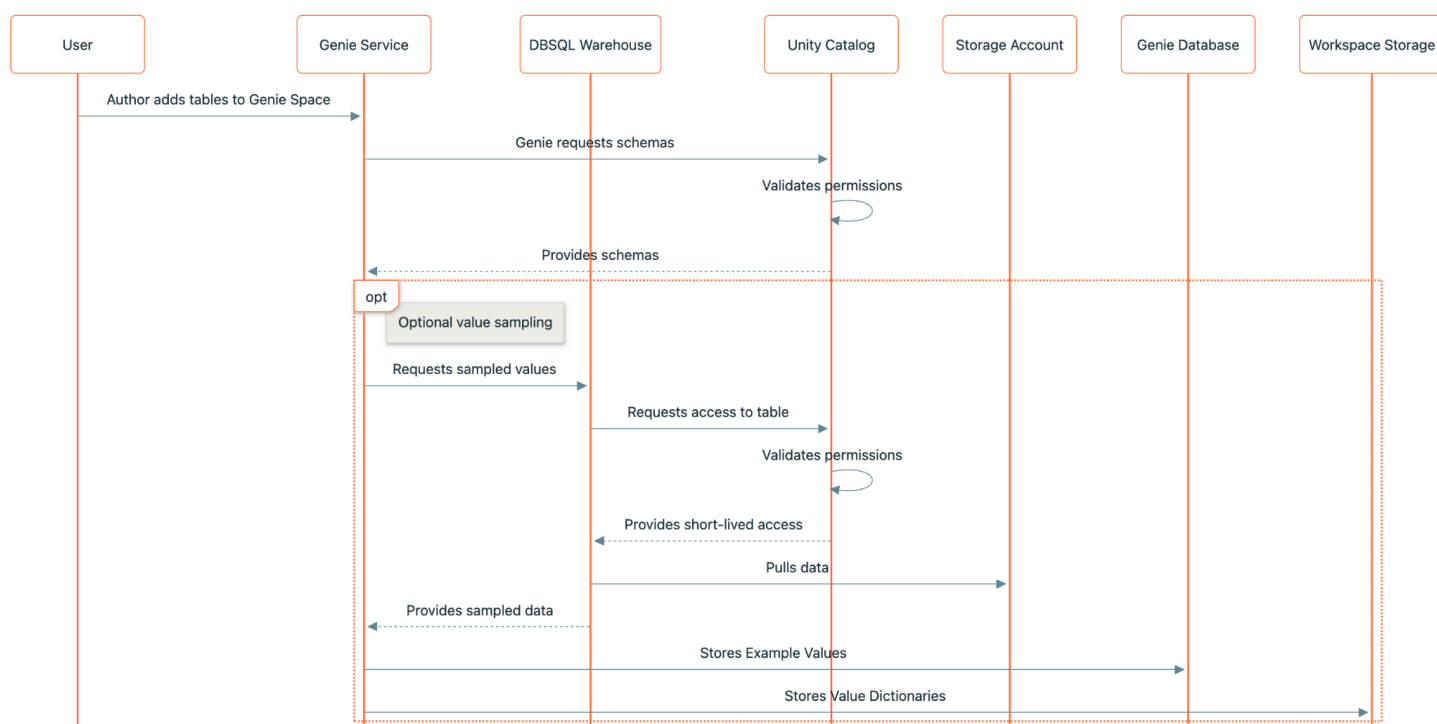
Today, we use Azure OpenAI as the LLM for Genie, but we consistently test alternative models, including from other frontier model providers, and may shift some or all Genie use to a better performing model at any time. Any model will adhere to the same overall security model (such as: no retention, no training).

The Genie service uses a control plane database to store application data required to operate the service, such as user questions and instructions (subject to customer-managed keys: managed services: [AWS](#), [Azure](#), [GCP](#)). The data used is similar to other AI features, [discussed above](#).

When a user runs a query in Genie, the SQL execution occurs on Databricks SQL warehouses, which store their results in the SQL result cache ([AWS](#), [Azure](#), [GCP](#)) in [workspace system data](#) so that they can be retrieved or recomputed when needed.

Below, find sequence diagrams for the Genie service. Functionality will evolve over time, but these are intended to broadly illustrate how the service operates.

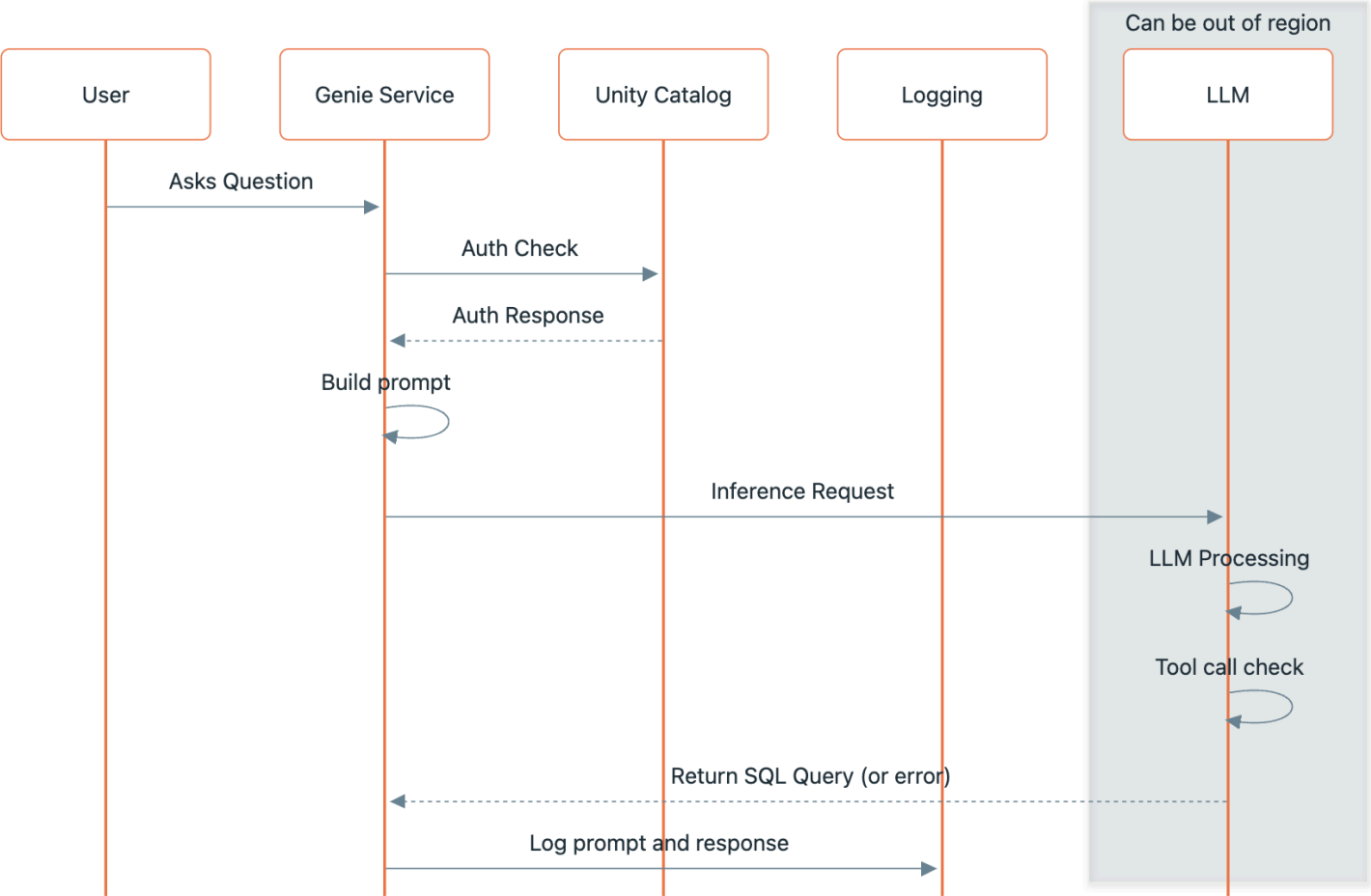
Adding Table to Genie Agent



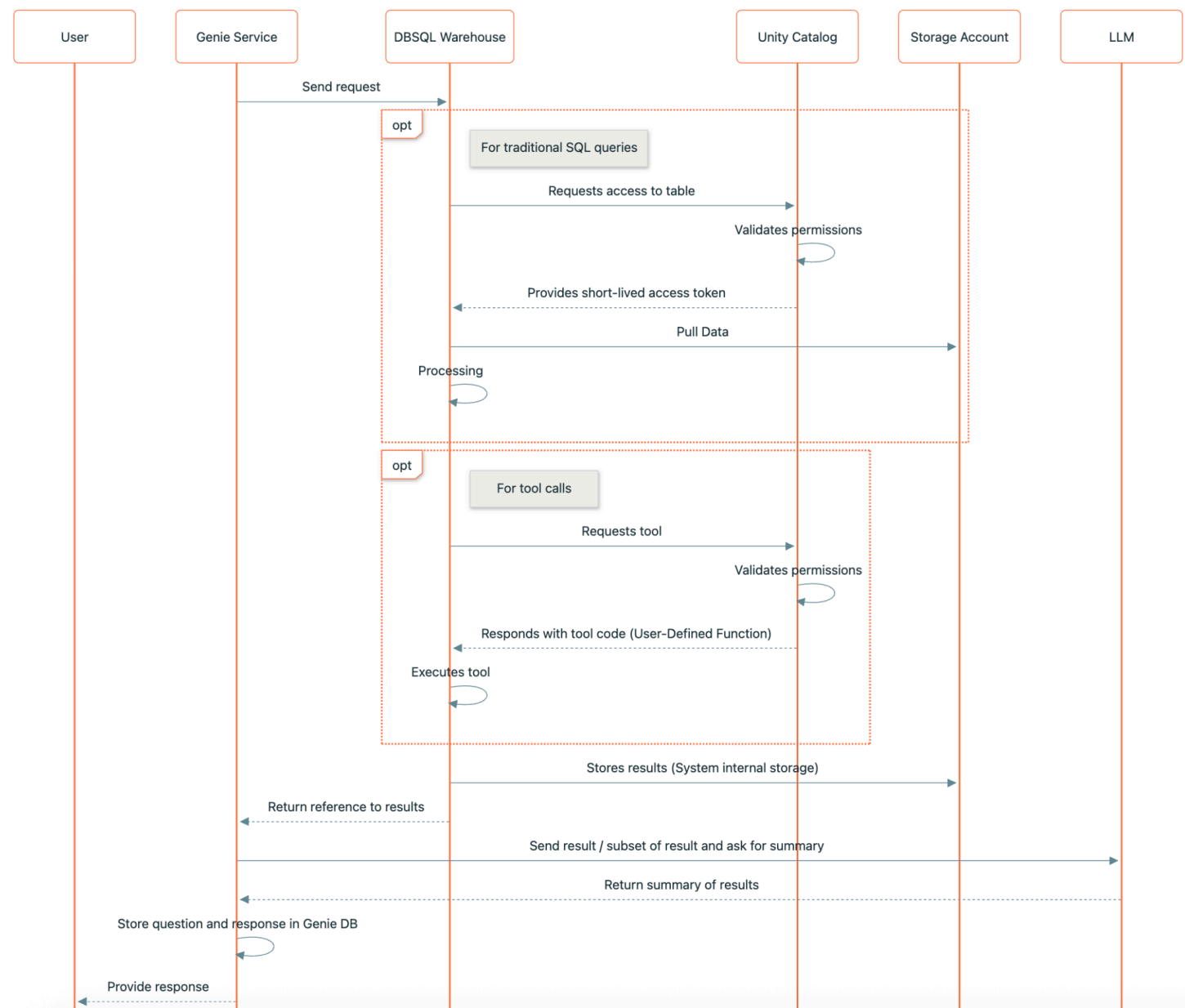
Genie Agents Query

Due to size, we have broken this out into two diagrams: sending the request to the LLM, and handling the SQL query from the LLM.

Send request to LLM



Handle SQL Query from LLM

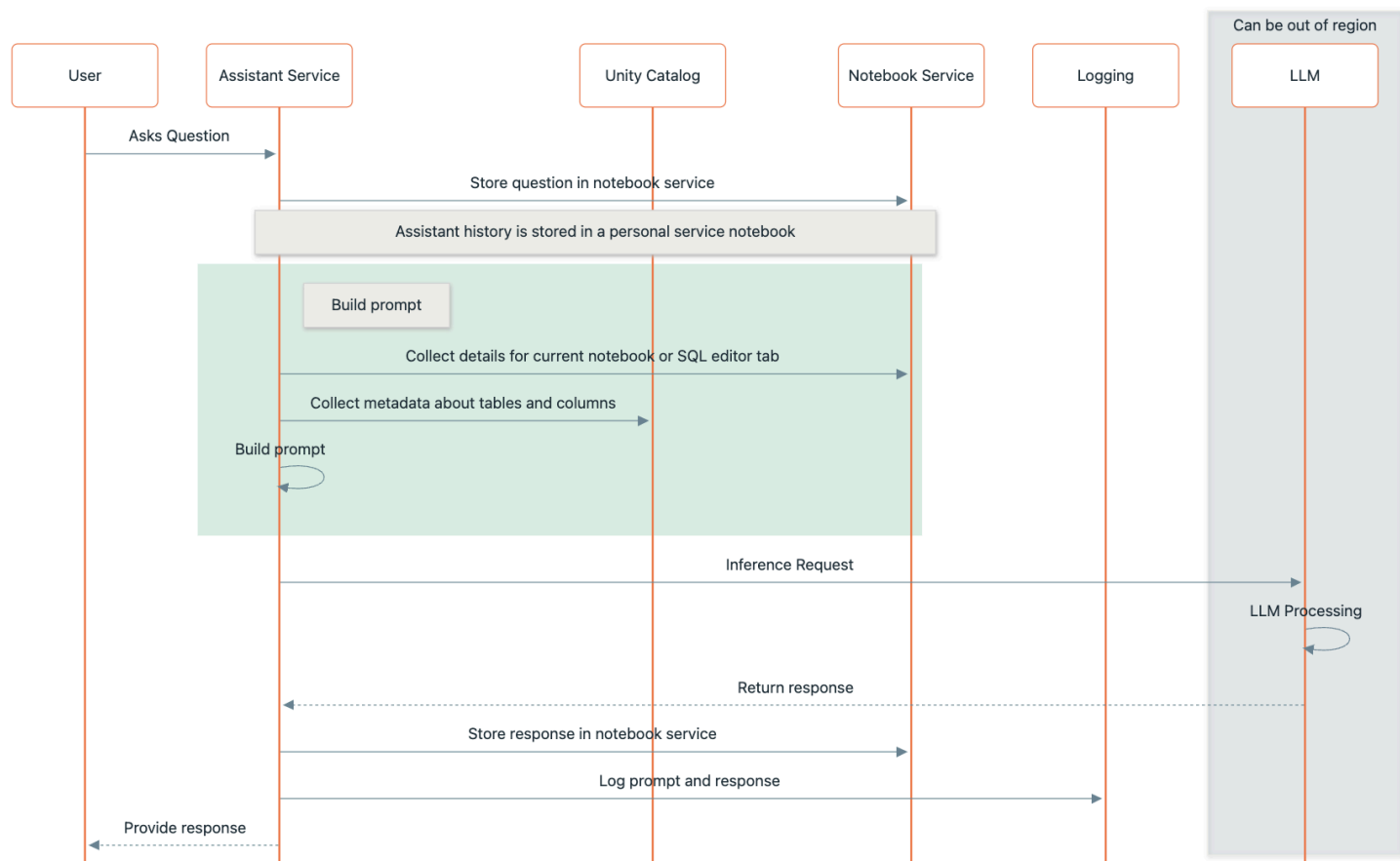


Genie Code

Genie Code (formerly known as the Databricks Assistant) takes user questions (or code issues) and retrieves answers from an LLM. The conversation history is stored in a special-use notebook only available to the user. Notebook and query chat histories aren't available to other users or admins, even if the query or notebook is shared. The data used is similar to other AI features, [discussed above](#).

Below, a sequence diagram for Genie Code. Functionality will evolve over time, but it is intended to broadly illustrate how the service operates.

Genie Code Question



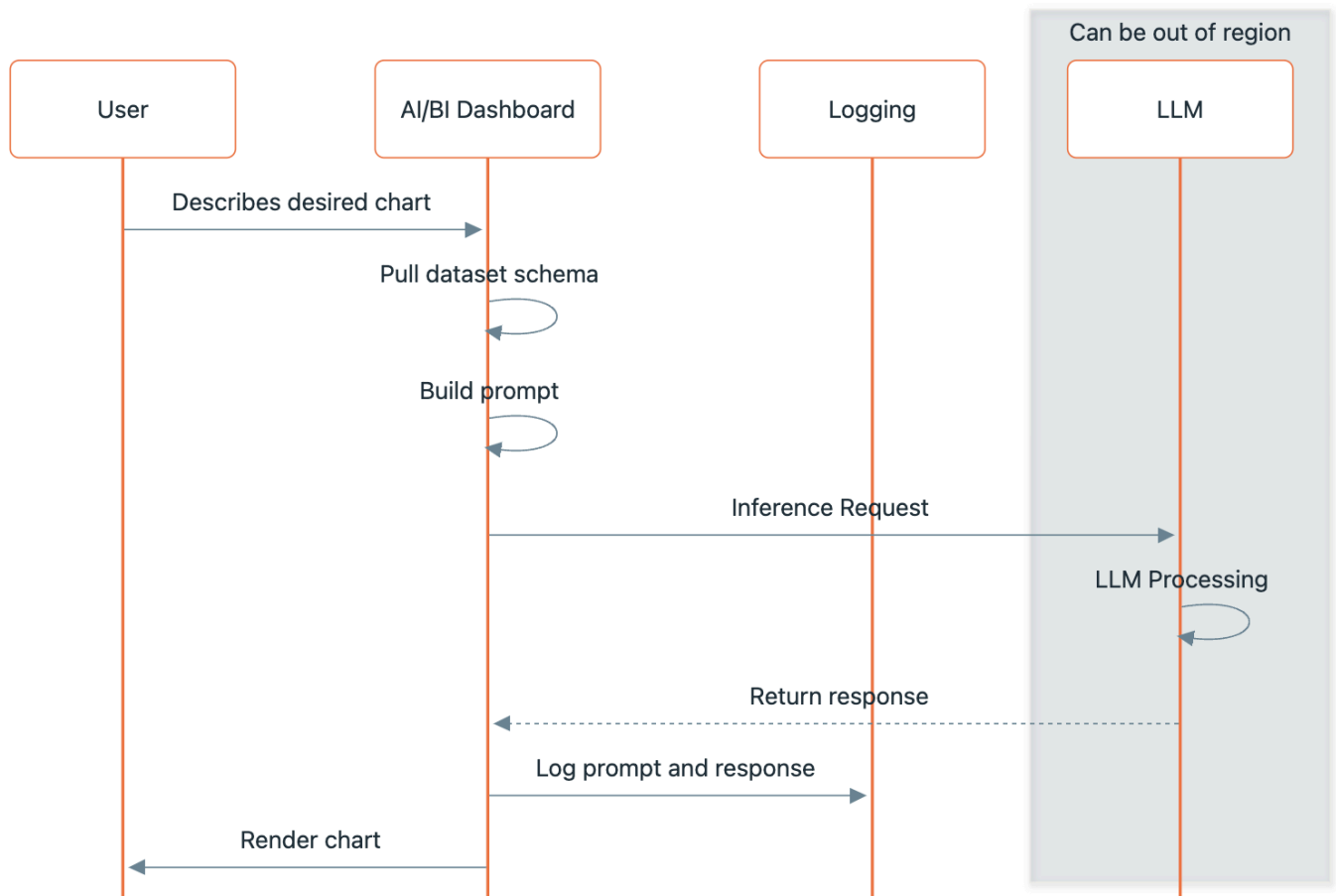
Note: The “personal service notebook” referenced is a user-specific notebook where conversations are stored. This notebook will not show in the UI and is private to each user, but otherwise governed like other notebooks.

AI/BI Dashboards

AI/BI Dashboards are the modern iteration of dashboarding within Databricks. In addition to the traditional dashboarding capabilities, they include AI-powered chart generation where the user can specify a dataset and describe the chart they want to see. AI/BI Dashboards are powered by Databricks SQL warehouses, and use an LLM (such as Azure OpenAI) to translate the user's description into our chart syntax. The data used is similar to other AI features, [discussed above](#).

Below, a sequence diagram for AI/BI dashboards. Functionality will evolve over time, but it is intended to broadly illustrate how the service operates.

Asking AI/BI Dashboard to generate a chart



Revision history

Date	Author	Description of changes	Version #
08/25/25	DV	First version	1.0
10/22/25	DV	Expanded example for the data used by AI features, updated models used	1.1
09/10/26	DV	Updated model hosting table to match live partner-powered AI documentation	1.2
09/10/26	DV	Added architecture section and shared responsibility model	1.3
09/15/26	DV	Switched Genie Spaces to Genie Agents	1.4